

Rtl66 Developer Guide 0.2.0

Chris Ahlstrom
(ahlstromcj@gmail.com)

January 25, 2026



Rtl66 Logo

Contents

1	Introduction	4
1.1	Rtl66: What!?	4
1.2	History	5
2	High-Level Description of Rtl66	5
2.1	Rtl Extras	5
2.1.1	Rtl Build Macros	6
2.1.2	Rtl rterror	6
2.1.3	Rtl api_base	6
2.1.4	Rtl rt_types	6
2.1.5	Rtl iothread	6
2.1.6	Rtl Test Helpers	6
2.2	Rtl MIDI	7
2.2.1	Rtl MIDI Base	7
2.2.1.1	Rtl MIDI Base midi_api	7
2.2.1.2	Rtl MIDI Base rtmidi	8
2.2.1.3	Rtl MIDI Base rtmidi_in	8
2.2.1.4	Rtl MIDI Base rtmidi_out	9
2.2.1.5	Rtl MIDI Base rtmidi_engine	9
2.2.1.6	Rtl MIDI Base Miscellaneous	9
2.2.2	Rtl MIDI Extensions	10
2.2.2.1	Rtl MIDI Ext. Bus	10
2.2.2.2	Rtl MIDI Ext. Bus-In	10
2.2.2.3	Rtl MIDI Ext. Bus-Out	10
2.2.2.4	Rtl MIDI Ext. Bus Array	11
2.2.2.5	Rtl MIDI Ext. Calculations Module	11
2.2.2.6	Rtl MIDI Ext. Client Info	11
2.2.2.7	Rtl MIDI Ext. Clocking	11
2.2.2.8	Rtl MIDI Ext. Event Codes	12
2.2.2.9	Rtl MIDI Ext. Events	12
2.2.2.10	Rtl MIDI Ext. Event List	12
2.2.2.11	Rtl MIDI Ext. File	12
2.2.2.12	Rtl MIDI Ext. Master Bus	12
2.2.2.13	Rtl MIDI Ext. Measures	13
2.2.2.14	Rtl MIDI Ext. Message	13
2.2.2.15	Rtl MIDI Ext. Midibytes	13
2.2.2.16	Rtl MIDI Ext. Player	13
2.2.2.17	Rtl MIDI Ext. Poller	13
2.2.2.18	Rtl MIDI Ext. Port	13
2.2.2.19	Rtl MIDI Ext. Port Naming	14
2.2.2.20	Rtl MIDI Ext. Ports	14
2.2.2.21	Rtl MIDI Ext. Scales	14
2.2.2.22	Rtl MIDI Ext. Splitter	14
2.2.2.23	Rtl MIDI Ext. Timing	14
2.2.2.24	Rtl MIDI Ext. Track	15
2.2.2.25	Rtl MIDI Ext. Track Data	15

2.2.2.26	Rtl MIDI Ext. Track Data Ex	15
2.2.2.27	Rtl MIDI Ext. Track Info	15
2.2.2.28	Rtl MIDI Ext. Track List	15
2.3	Rtl Audio	16
3	Summary	16
3.1	Summary / RtMidi API	17
3.2	Summary / Masterbus API	17
4	Walk-throughs	17
4.1	Walk-throughs / RtMidi API	18
4.1.1	Walk-throughs / RtMidi API / apinames	18
4.1.2	Walk-throughs / RtMidi API / qmidiin	18
4.1.3	Walk-throughs / RtMidi API / cbmidiin	20
4.1.4	Walk-throughs / RtMidi API / midiclock	20
4.1.5	Walk-throughs / RtMidi API / midiout	20
4.1.6	Walk-throughs / RtMidi API / midiprobe	20
4.1.7	Walk-throughs / RtMidi API / sysextest	20
4.1.8	Walk-throughs / RtMidi API / test_c_api	21
4.2	Walk-throughs / Master Bus API	21
4.2.1	Walk-throughs / Master Bus API / midiprobeex	21
4.3	Supported MIDI APIs	21
5	MIDI Meta and System Exclusive Events	21
5.1	MIDI Meta Events	21
5.1.1	Generic Meta Messages	22
5.1.2	Track/Sequence Number (0x00)	23
5.1.3	Track/Sequence Name (0x03)	23
5.1.4	End of Track/Sequence (0x2F)	23
5.1.5	Set Tempo Event (0x51)	23
5.1.6	Time Signature Event (0x58)	24
5.1.7	Key Signature Event (0x59)	25
5.1.8	Text Events	25
5.2	Sequencer-Specific Meta-Events Format	26
5.2.1	Meta Events Handling	26
5.3	System Exclusive Event (0xF0)	27
5.3.0.1	Single SysEx Message	28
5.3.0.2	Continuation Events	28
5.3.0.3	Escape Sequence	29
5.4	System Exclusive Sample Dump Standard	29
5.4.0.1	Generic Handshaking Messages	29
5.4.0.2	Data Messages	30
5.4.1	Universal SysEx Events (0xF0 0x7E and 0xF0 0x7F)	31
5.4.1.1	Rtl66 SysEx Handling	31
6	References	32

List of Figures

1	Rtl66 Overview	16
---	--------------------------------	----

List of Tables

1 Introduction

This document describes "Rtl66", an audio and MIDI library based on the *RtAudio* [7] and *RtMidi* [8] libraries, with changes and extensions. The Audio library is not at all ready for prime time. The MIDI library is complete in its design, but only JACK and ALSA are basically complete at this time. The following project supports *Rtl66* (and documentation):

- <https://github.com/ahlstromcj/rtl66.git>
- <https://ahlstromcj.github.io/>

Feel free to clone or fork it!

1.1 Rtl66: What!?

The *Rtl66* library is meant to support a potential *Seq66v2* application, as well as other audio/MIDI applications. It is based on the *RtMidi* [8] and *RtAudio* [7] libraries, but with the following changes and additions:

- New naming conventions. Not a big fan of camel cases, so class and function names have been converted to use lower case and underscores.
- Modularization. Rather than a monolithic C++ file, *RtMidi.cpp/h* has been broken into a couple dozen separate files so that it is easier to zero in on a particular class or API.
- Additional support functions. For example, a simple command-line parser and various test function have been added to support the test applications in a uniform way.
- Pipewire. In progress, this API will be added and will be supported in *Linux*.
- Library code based on *Seq66*. This includes simplified versions of MIDI busses, calculations, port information, JACK transport, auto-connect, and more. This code can be adapted more easily for use by other developers. There are more such modules to be completed in the future.
- Support for a "master bus". *RtMidi* supports each MIDI port having a separate client. The *Rtl66* master bus concept uses the same client for all ports, which has important implications for processing.

The original (but refactored) *RtMidi* code resides in the `include/rtl` and `src/rtl` directories. The "C" interface has been preserved to a large extent, but the "C++" uses namespaces and modified names. It also provides additional paradigms.

The Seq66-based additions to the library are in the `include/midi` and `src/midi` directories. In the future, there will be a *Seq66* library that relies on the *Rtl66* library and adds concepts like performer, triggers, playlists, etc.

There is also a `transport` directory, currently containing only a JACK transport class. Perhaps we can add *Ableton Link* [3] transport later.

At some point we might add rudimentary audio support, for playing short clips in MIDI files.

1.2 History

Well into the development of *Sequencer64* [10], we decided to add support for JACK MIDI. After a quick look around, *RtMidi* seemed to be the best basis for JACK MIDI. However, after getting well into development, it was found that the *RtMidi* model and the *Sequencer64* models did not align well.

The result was a butchery of the *RtMidi* library, much added complexity to the application libraries, and a difficult-to-extend application. Furthermore, the Mac/Windows support was dropped, and instead the existing *PortMidi* [6] support was used for those operating systems.

With a big laundry list of potential upgrades, and a desire to simplify *Seq66*, it was decided to fold *Seq66* code into our refactored version of *RtMidi* as well.

2 High-Level Description of Rtl66

The *Rtl66* library refactors the *RtMidi* and *RtAudio* libraries, and then extends them with additional support for MIDI and audio operations. (Audio support is not at all ready at this time). Some additional helpful modules (e.g. for I/O threads) have also been created.

Some of these items are directly copied from the *Seq66* project, with minor modifications (coding conventions, repartitioning) or additions such as `detect_jack()` or `find_midi_api()`.

2.1 Rtl Extras

The "extras" are modules in the top-level directories `include/rtl` and `src/rtl`. They provide functionality common to MIDI and audio support. Additions to *RtMidi* are support for I/O threads and a test-helpers module.

MIDI support resides in `include/rtl/midi` and `src/rtl/midi`.

Audio support resides in `include/rtl/audio` and `src/rtl/audio`.

They both reside in the `rtl` namespace. In the subdirectory `rtl/midi` are subdirectories for ALSA, JACK, Windows MM, Pipewire, and more. Only ALSA and JACK are fully developed at this time.

In addition, MIDI support adapted from *Seq66* resides in `include/midi` and `src/midi` and are in the `midi` namespace.

2.1.1 Rtl Build Macros

The `include/rtl/rtl_build_macros.h` header file contains macro definitions for version information, code export settings, various features, and for the creation of the supported APIs. The latter are somewhat determined by the build platform (e.g. *Windows* versus *Linux*).

In the future, some feature macros might be migrated to the `meson.options` file.

2.1.2 Rtl rterror

The `rterror` class provides an enumeration to indicate the category of error (e.g. a driver error versus a warning), an alias for an error callback function, and some virtual message-related functions.

2.1.3 Rtl api_base

The `api_base` class currently provides only support for the handling of errors and warnings. It provides the ability for a caller to set a callback function to handle errors, including a pointer to error-callback data. Free functions for printing messages are provided.

2.1.4 Rtl rt_types

This C++ header file simply contains the following lines:

```
#include "rtl/midi/rt_midi_types.hpp"
#include "rtl/audio/rt_audio_types.hpp"
```

2.1.5 Rtl iothread

The `iothread` class supports starting an `std::thread` that will run a caller-supplied function object (also known as a *functor*). It also simplifies launching and joining a thread. For an example of usage, see the `midi::player` class and the `play` test program.

2.1.6 Rtl Test Helpers

Provides some free functions in the `globalnamespace`. They provides some simple functions to choose test I/O port numbers, show basic help text, handle options, brief sleep times, and some other basic settings.

To show all options, use the `-help` option. Note that not all options are supported by all test applications.

2.2 Rtl MIDI

The *Rtl66* library consists of the core of *RtMidi* and extensions to it. The core is in the `rtl` namespace and subdirectory, while the extensions are mostly in the `midi` namespace and subdirectory.

2.2.1 Rtl MIDI Base

This section describes the classes and modules derived from the *RtMidi* library. Rather than a monolithic module, the classes have been split across multiple modules for easier management. They reside in the `rtl` namespace.

Modules to support many MIDI protocols have been created, but currently only the ALSA and JACK APIs have been refactored and tested.

2.2.1.1 Rtl MIDI Base `midi_api`

The `midi_api` class is an *abstract base class* ("ABC") for all of the various MIDI OS-related class (ALSA, JACK, MacOSX Core MIDI, etc.). The actual API in use is selected from one of the compiled-in APIs, which depends on the build platform.

The `rtmidi` class is assigned a pointer to an instantiated MIDI API class, and all the MIDI calls are done through this pointer.

The functionality covered is:

- *Port type*. Indicates if the port is for input, output, duplex (both input and output), finder, and engine.
 - `midi::port::io::input`. The port is for input.
 - `midi::port::io::output`. The port is for output.
 - `midi::port::io::duplex`. The port is for input or output. For JACK, this selects a special JACK process function.
 - `midi::port::io::engine`. Engine is a concept whereby only one MIDI client needs to be created, and the ports can all use this common client. For JACK, this selects a special JACK process function. A single client has implications for performance and for multi-threading.
 - `midi::port::io::dummy`. A non-functional "port". Might be useful for testing, or for flagging a lack of MIDI support.
 - `midi::port::io::finder`. Finder means that no connections are to be made. Instead, the only task is to find a useable MIDI API.
- *MIDI input data*. The `rtl::rtmidi_in_data` class provides data such as an input queue, an arbitrary API data buffer, input callback support, an input buffer, and some boolean flags. These handles are void pointers to API data or user data. The functions handling these pointers must assume the correct data type. Callers can optionally provide a callback function to handle MIDI input.
- *Master buss*. The `midi::masterbus`, if created, provides the common MIDI client data for the MIDI "engine". If not created, operation is much like that of the *RtMidi* library.

- *Engine connection/disconnection.* If a **masterbus** was created, a connection is stored as a void pointer, as each port will use the master's client handle and other data. Compare the RtMidi style in the test applications "midiout", "cbmidiin", and "qmidiin" with the master-bus style test applications "busout" and "busin".
- *Port enumeration.* Virtual functions are provided to find all existing MIDI ports, count them, and get their names. For JACK, one or two port aliases can also be obtained that are human readable.
- *Client activation/port-opening/closing.* For the individual port under the original "RtMidi" scheme, an API client (what we term "the engine") is made and activated (in JACK), the port is opened, used, and then closed.
- *Sending.* Virtual functions for various send operations are provided. Non-functional in this base class.
- *Receiving.* Virtual functions for polling and event-retrieval operations are provided. Non-functional in this base class.
- *Miscellaneous.* Various other features are provided. Unnecessary to describe them here.

2.2.1.2 Rtl MIDI Base `rtmidi`

The `rtmidi` class is an abstract base class with non-virtual access functions for the `midi_api` functions, plus some static functions. The functions to open a regular and a "virtual" port are virtual just so the default port name can be set correctly.

The enumeration `rtmidi::api` reflects the APIs available (though some are not yet written); the pure-virtual function `open_midi_api()` is written in `rtmidi_in`, `rtmidi_out`, and `rtmidi_engine`.

A number of functions are non-virtual and non-static. They forward calls to the virtual functions of the current API object, accessed via `rt_api_ptr()`. The actual functions of the selected API are called through the stored `midi_api` pointer, if it exists, as in the following implementation of `flush_port()`:

```
bool rtmidi::flush_port ()
{
    return rt_api_ptr()->flush_port();
}
```

Also defined are a number of static functions to gather information about the supported APIs and to select the API to be used for the run of an application.

Note that `rtmidi` does not support the *Seq66/Rtl66* concept of a "master bus", though it is used in that paradigm.

2.2.1.3 Rtl MIDI Base `rtmidi_in`

The `rtmidi_in` class extends `rtmidi` to add functions for setting the input buffer size, setting or canceling the input callback function, ignoring MIDI data such as SysEx, and a function to get `midi::message` data. It relies on the base-class functions for some of the work.

It also overrides `open_midi_api()` to try to open one of the MIDI APIs for input.

An `rtmidi_in` is provided as a member of `midi::bus_in` and using in pass-along functions to get port information, poll for MIDI events, get `midi::messages`, and access the pointer to the MIDI API object.

Should we reconsider and derive from, rather than contain, `rtmidi_in`?

2.2.1.4 Rtl MIDI Base `rtmidi_out`

It also overrides `open_midi_api()` to try to open one of the MIDI APIs for output. It relies on the base-class functions for some of the work.

An `rtmidi_out` is provided as a member of `midi::bus_out` and using in pass-along functions to get port information, send bytes, send `midi::events`, send `midi::messages`, and access the pointer to the MIDI API object.

Should we reconsider and derive from, rather than contain, `rtmidi_out`?

2.2.1.5 Rtl MIDI Base `rtmidi_engine`

The `rtmidi_in` class extends `rtmidi` to add a pointer to a `midi::masterbus`.

It also overrides `open_midi_api()` to try to open one of the MIDI APIs as an "engine" (neither input nor output). If successful, then the `midi::masterbus` pointer is passed to the currently-selected MIDI API object.

2.2.1.6 Rtl MIDI Base Miscellaneous

This section briefly describes some additional modules.

- `find_midi_api`. Provides free functions to find and try to open a MIDI API object. Some platforms provide multiple potential APIs and a fall-back API in case the caller does not specify one. For example, on *Linux*, JACK is the fallback API if detected, otherwise ALSA is the fallback API.
- `midi_dummy`. This API is a non-functional class to be used only when necessary, perhaps in testing.
- `midi_queue`. Provides a queue of `midi::message` objects. It is used for getting input events. It provides functions for push/pop of events, allocating data space, and retrieving events.
- `rtmidi_in_data`. Includes a `midi_queue` for incoming data, plus callbacks, optional user-specific data, buffer size, API data, and other book-keeping for MIDI input events.
- `rt_midi_types`. Currently empty. See `rt_types.hpp` instead.
- `rtmidi_c`. Provides C-function wrappers for `rtmidi` functionality. Also provided are plain C enumerations corresponding to `rtl::rtmidi::api` and `rtl::rterror::kind`.
- *MIDI API directories*. Directories provided are: `alsa`, `jack`, `macosx`, `pipewire`, `webmidi`, and `winmm`. Their contents will be discussed elsewhere. Currently, only `alsa` and `jack` are fully implemented. Also, there are some newer APIs not yet represented.

2.2.2 Rtl MIDI Extensions

They reside in the `midi` namespace. Let's get the tricky "buss" stuff out of the way first.

2.2.2.1 Rtl MIDI Ext. Bus

The `midi::bus` class is base/mixin class for providing *Seq66* concepts for `midi::bus_in` and `midi::bus_out`. It is tied to the "master bus" paradigm where a single MIDI API ("engine") client is used to handled all ports, as busses. It provides the following concepts:

- *Master Bus*. Like `rtl::rtmidi_engine`, this class hooks in a `midi::masterbus`.
- *Bus Index*. This value is an ordinal number starting at 0. In an application, MIDI busses are referred to by this number, not by name. This value, along with the values discussed below, is stored in a `midi::port` structure.
- *Client ID and Display Name*. The client number determined by the MIDI engine, if available. The display name is assembled by the `set_name()` function.
- *Bus ID and Bus Name*. The buss number determined by the MIDI engine, if available. In ALSA, these two are the "client:bus" (e.g. "14:0") numbers. The bus name is obtained by `ports::get_bus_name()`. (Note: sometimes our documentation confuses port versus buss.)
- *Port Number, Port Name, and Port Alias*. The name is accessed by `get_port_name()` and the alias is accessed by `get_port_alias()` (useful only with JACK).
- *Port Type, Port Kind*. The type indicates if the bus is for input or output. The kind indicates if it is a system, virtual, or normal port.
- *Clock type*. Indicates both the port status (unavailable, disabled) and the sending of MIDI Clock and Start.
- *Mutex*. A recursive locking mutex that can be used in processing.
- *MIDI Processing Functions*. Provides function to activate/deactivate, start, send system exclusive, controlling MIDI Clock.
- *Other Functions*. There are functions to access all of the properties of the bus.

2.2.2.2 Rtl MIDI Ext. Bus-In

This class is derived from `midi::bus` and contains an `rtl::rtmidi_in` object to implement getting MIDI messages from an external source.

See section [2.2.1.3 "Rtl MIDI Base rtmidi_in"](#), page 8.

2.2.2.3 Rtl MIDI Ext. Bus-Out

This class is derived from `midi::bus` and contains an `rtl::rtmidi_out` object to implement getting MIDI messages from an external source.

See section [2.2.1.4 "Rtl MIDI Base rtmidi_out"](#), page 9.

2.2.2.4 Rtl MIDI Ext. Bus Array

The `midi::busarray` contains and manages an array of `midi::bus` objects. Two of these objects are used by the master bus, one for input, and one for output.

It is implemented by an internal `midi::busarray::container` object. It holds a vector of pointers to `midi::bus` objects. It provides functions to access a bus and do things with the MIDI port that it wraps.

The `midi::busarray` object uses the "pimpl" paradigm [5] where its functions use a pointer to the "private" `midi::busarray::container` to do the work. Pimpl was used because of an issue with the dependence on the bus class.

2.2.2.5 Rtl MIDI Ext. Calculations Module

The `calculations` module provides enumerations and functions to centralize various calculations needed by a MIDI program.

The enumerations provided support snapping time, generating waveforms, performing alterations such as quantizing,

The functions, many of them inlined, support standard MIDI values for timing, PPQN, grid measurements, time-scaling, pitch and tempo calculations, and much more.

2.2.2.6 Rtl MIDI Ext. Client Info

This module contains two big items: a client defaults structure and a client-info class. It also defines a small input-specification structure.

The `midi::client_defaults` structure contains members to specify client and application name, API version, preferred MIDI API, thread and port usage, and more. The application should make an instance of this structure and initialize it. See the "busin" test application, for example.

The `midi::input_specs` specifies if the input is active, uses SysEx, time code, active sensing, an optional user callback and optional data. See the "busin" test application, for example.

The `midi::clientinfo` class contains both of those structures with constructors that can get those structures. The class adds current and previous `midi::ports`, saves a pointer to the client (engine) handle, status flags, and an error string.

The client-info object is later filled with a list of the existing MIDI ports. Some of the items loaded into this object can potentially be saved to a configuration file.

2.2.2.7 Rtl MIDI Ext. Clocking

The `clocking` module contains *Seq66* concepts for MIDI clocking. There are enumeration values for starting/stopping MIDI clock, the clocking and input/output status of a MIDI port, and some functions to check the clocking values.

2.2.2.8 Rtl MIDI Ext. Event Codes

The `eventcodes` module somewhat duplicates the "EVENT_XXX" macros in the `seq66::event` class, but instead uses enumerations in the `midi` namespace, and functions to deal with them.

The `midi::status` enumeration class defines note off, note on, and other statuses.

The `midi::meta` enumeration class defines the MIDI meta events such as track number, time/key signatures, text events, and more.

The `midi::ctrl` enumeration class defines the MIDI control events such as volume and balance.

Many conversion functions are provided to convert between the enumeration values and simple data types; to determine what a byte actually means; to get the human-readable name for the event, RPN conversions, and more.

2.2.2.9 Rtl MIDI Ext. Events

The `midi::event` class is much like `seq66::event`, but it also uses the `eventcodes` module, and is more "C++" than the original. Instead of a byte array, it holds a `midi::message`. See section [2.2.2.14 "Rtl MIDI Ext. Message"](#), page 13.

It is still *not* broken into a class hierarchy of events, something that we worry about.

2.2.2.10 Rtl MIDI Ext. Event List

The `midi::eventlist` class is much like `seq66::eventlist`. It has enumerations for selecting and editing events. It has many functions for editing the list, storing events, getting the events, iterating through events, and all of the other modifications supported by *Seq66*.

2.2.2.11 Rtl MIDI Ext. File

The `midi::file` class recreates the basic part of the `seq66::midifile` class. It deals with `midi::tracks` instead of "sequence" objects, but not sequencer-specific events.

2.2.2.12 Rtl MIDI Ext. Master Bus

The `midi::masterbus` class is similar to that concept in *Seq66*, but does not (at this time) require deriving a child master bus.

It supports having a single MIDI API client handle (an "engine") that is used by all of the ports in the input/output `midi::busarray` objects. It holds a void pointer to the client handle. It holds a `clientinfo`.

Normally, using the basic "rtmidi" process requires creating a new client handle for each port.

On the other hand, the `midi::masterbus` creates the client handle that all of the ports use, and manages the whole setup process. The use of a single client handle has implications for some of the

processing.

Examples in the `tests/midi` directory are the `busin`, `busout`, `play` test applications. In the future, we will experiment with processing such as round-robin port selection, to improve latency.

2.2.2.13 Rtl MIDI Ext. Measures

The `midi::measures` class is a small class to manage beats, bars, and divisions (data type `midi::pulse`).

2.2.2.14 Rtl MIDI Ext. Message

The `midi::message` class contains a static counter for debugging, a floating-point time-stamp, a container of `midi::bytes`, an optional channel number, an optional MIDI buss index number, and an optional MIDI event type (an ALSA concept).

It holds all of the bytes for a message, so saving the event or writing it is very simple (unless applying a channel)... just dump the bytes.

2.2.2.15 Rtl MIDI Ext. Midibytes

The `midibytes` module is basically the same module copped from *Seq66*. It defines, in the "midi" namespace, aliases for integer, boolean, and byte types, plus a vector called `midi::bytes`. It defines functions to test bytes, convert to a pointer, clamp values to byte-size, and more.

2.2.2.16 Rtl MIDI Ext. Player

The `midi::player` class can read a simple MIDI file and play it. It accepts a reference to a `midi::masterbus`, and provides an output thread. It can also poll for MIDI events. It's not complete, but is meant for testing and experimenting. It includes a nested "synch" class for managing a condition variable to control output. It uses an output thread function. See the "play" test app.

2.2.2.17 Rtl MIDI Ext. Poller

The `midi::poller` class, like `midi::player`, is a provisional class for testing and experimentation, except that it polls for data rather than sending MIDI events.

It includes an `xpc::fifo<midi::message>` (from the *xpc66* library project) and `std::unique_ptr<inqueue>` to store the incoming data. It includes a nested "synch" class for managing a condition variable to control input. It uses an input thread function. See the "busin" test app.

2.2.2.18 Rtl MIDI Ext. Port

The `midi::port` class is a data class that describes information about a port, including the system numbers and names of the port, the index of the port in the application, alias names (if applicable) for the port, the client number, whether retrieved from the system or made up, is the "buss" number.

Also defined are the `midi::port::io` and `midi::port::kind` enumerations. The former indicates what the port is meant for; see section [2.2.1.1 "Rtl MIDI Base `midi\_api`", page 7](#). The latter indicates if the port is to be a normal, manual (i.e. virtual), or system port. Also see section [2.2.2.6 "Rtl MIDI Ext. Client Info", page 11](#).

2.2.2.19 Rtl MIDI Ext. Port Naming

The `midi::portnaming` module is meant to extract port-naming information from system port names or aliases. The `midi::portnaming` enumeration class provides for styles of brief, pair, and full naming styles. Functions are provided to extract the client/buss name, the port name, names created by the `a2jmidid` daemon, JACK alias names, and short names called "nicknames" which are cleaner than full port names and can be useful in port-mapping (as in *Seq66*).

2.2.2.20 Rtl MIDI Ext. Ports

The `midi::ports` class provides a container of `midi::port` objects of the same type (input, output). The default port type is `midi::port::io::dummy`.

Functions are provided to add port data to the container, access port data by buss index, etc. Many of these functions simply look up the `midi::port` object and retrieve port data members.

2.2.2.21 Rtl MIDI Ext. Scales

The `midi::scales` module provides enumeration classes for musical keys, musical scales, and chords. Functions are provided to convert between these enumeration values and integer values and to validate values. Functions are provided to get human-readable names for the scale, transpose the scale up or down, etc. These functions are useful for selecting a key, a scale, and a chord from a GUI element.

2.2.2.22 Rtl MIDI Ext. Splitter

The `midi::splitter` class provides a way to split an SMF 0 `midi::track` into a number of tracks on a per-channel basis. The resulting tracks are installed into a `midi::player` object.

2.2.2.23 Rtl MIDI Ext. Timing

The `midi::timing` module and class holds information and limits about time signatures, PPQN, clocks/metronome, 32nds/quarter, and microseconds/quarter-note.

The most important functions related to beats/bar (BPB), beat width (BW), pulses per quarter-note (PPQN), and beats/minute (BPM).

Also see section [2.2.2.27 "Rtl MIDI Ext. Track Info", page 15](#).

2.2.2.24 Rtl MIDI Ext. Track

The `midi::track` class is a cut-down version of the `seq66::sequence`. It has an enumeration to determine how recording is to work. It has a pointer to its owning ("parent") `midi::player` object. It holds a `midi::trackinfo` object, a `midi::trackdata` object, an `xpc::recmutex`, a `midi::masterbus`, and a number of attributes describing the tracks.

2.2.2.25 Rtl MIDI Ext. Track Data

The `midi::trackdata` class holds a partial set of data for a track. It contains a `midi::eventlist`, a `util::bytevector`, and some other status. The `util::bytevector` is a class that handles big-endian data; MIDI is encoded in big-endian.

The `midi::trackdata` class provides a number of "get" functions to get values and parts of MIDI events from bytes, plus a number of "put" functions to convert basic data items and MIDI events to bytes, plus a number of "extract" functions to extract MIDI events from the current location in track data.

2.2.2.26 Rtl MIDI Ext. Track Data Ex

The `midi::trackdataex` class is smaller than `midi::trackdata`, and works with "triggers". However, it is not compiled into the build, and is a work in progress.

2.2.2.27 Rtl MIDI Ext. Track Info

The `midi::trackinfo` module provides supplemental classes to group data items and simplify function calls by using data structures. It defines the following classes:

- **tempo**info. The members specify a tempo track number, beats/minute, and microseconds/quarter-note. Compare it to the `midi::timing` class (section [2.2.2.23 "Rtl MIDI Ext. Timing"](#), page [14](#)).
- **timesig**info. The members specify beats/bar, beat-width, clocks/metronome, and 32nds/quarter-note. Compare it to the `midi::timing` class (section [2.2.2.23 "Rtl MIDI Ext. Timing"](#), page [14](#)).
- **keysig**info. The members specify the name of the key-signature, the sharp/flat count, and a flag for a minor scale.
- **track**info. Defines the track name, its length, the three classes above in this list, an optional channel override, and a flag for being exportable (to be explained elsewhere).

2.2.2.28 Rtl MIDI Ext. Track List

The `midi::tracklist` class provides a vector of pointers to `midi::track` objects, plus the modified and sorted statuses.

They reside in the `audio` namespace. TBD. Not yet started.

Now let's take a quick overview of the two paradigms: the original paradigm and the masterbus paradigm. Here is a UML-like diagram of the main classes.



In this diagram, the classes in the `rtl` namespace mostly The classes in the `midi` namespace add a master-buss with a single API client handle that supports multiple busses (i.e. ports).

16

3.1 Summary / RtMidi API

The basic operation of an "rtmidi" application is to create an `rtl::rtmidi_in` or `rtl::rtmidi_out` object. These have access to common MIDI API and I/O functions from the `rtl::rtmidi` base class. They also create the appropriate MIDI API object, such as a `rtl::midi_alsa` object, and then open the associated port.

The `rtl::rtmidi` functions forward operations to the `rtl::midi_api` object accessed via a pointer. (This linkage is not shown in that diagram.)

The one exception to the RtMidi paradigm here is the `rtl::rtmidi_engine` class. It is used in the master-buss paradigm. It is used by the master buss, which sets up the MIDI client handle and a port to handle the paradigm described in the next section.

In any case, the result of the RtMidi paradigm is a number of ports enumerated from the system, each with it's own independent MIDI client handle. In ALSA, each port calls a MIDI-ALSA-handler function, passing the client handle to it via an ALSA data structure.

3.2 Summary / Masterbus API

The master-buss API is very similar to the API used by *Seq66*. It has implications for both JACK and ALSA, as noted below.

The basic operation of a "masterbus" application is to create a `midi::masterbus`. It uses an `rtl::rtmidi_engine` object to do the work. The `rtl::rtmidi_engine` object to do the work. It opens the MIDI API, sets up the appropriate I/O callback, if applicable.

The "master" then takes care of enumerating the ports that exist in the system, creating `midi::bus_in` and `midi::bus_out` objects for each port.

These "busses" are entered into an input or an output `midi::busarray`, where they can be polled or handle sending MIDI events.

For ALSA, as noted in the previous section, each port calls the MIDI-ALSA-handler function. However, the ALSA API is *not* thread-safe. Therefore, the masterbus must either process the input ports in some serial fashion, or use an input handler protected by a mutex. Currently, serial processing is used.

For JACK, one issue with the master-buss paradigm is that output is ragged as the JACK frame size is increased. This issue is something we will start testing and see how it can be solved.

4 Walk-throughs

Let's walk through the main test applications. There are two categories of tests: tests of the basic RtMidi-like API, and tests of the masterbus paradigm.

The RtMidi API:

- `apinames`
- `qmidiin`
- `cbmidiin`
- `midiclock`
- `midiout`
- `midiprobe`
- `sysextest`
- `test_c_api`

The masterbus API:

- `busin`
- `busout`
- `midiprobeex`
- `play`
- `smoke`

4.1 Walk-throughs / RtMidi API

The RtMidi API is encapsulated in the `rtl::rtmidi` namespace.

4.1.1 Walk-throughs / RtMidi API / `apinames`

This test gets the compiled APIs (e.g. in Linux ALSA and JACK are compiled in) and the detected APIs (ALSA will generally always be found, but JACK requires the user to start it).

The lists are `rtl::rtmidi::api_list` items. `rtl::rtmidi::get_compiled_apis()` and `rtl::rtmidi::get_detected_apis()` fill the lists. `rtl::rtmidi::show_apis()` shows the lists.

Note that there are two names for the API: a short internal name and a better name for display. For example `web_midi` and `Web MIDI` API. The functions are `rtl::rtmidi::api_name()` and `rtl::rtmidi::api_display_name()`. Also tested is `rtl::rtmidi::api_by_name()` to look up the API via its short name.

The C interface prepends "rtmidi_" to those functions, and of course there is no namespace scope.

4.1.2 Walk-throughs / RtMidi API / `qmidiin`

`tests/qmidiin` supports querying devices and then getting MIDI events via a `midi::message` or a `midi::event`, based on defining a macro in the source code. In addition, there is an option to read input from one MIDI source, or all MIDI sources.

`tests/qmidiin`. We run our debugger, set a breakpoint where the input object is created, and run using ALSA.

```
$ cgdb build/tests/qmidiin
(gdb) r --alsa
```

If JACK is not running, the option `-alsa` is not needed, as the library will fall back to ALSA (on Linux). Here is what happens:

1. Select ALSA as the API.

- Use the command-line to set the `-alsa` option.
- `rt_simple_cli()` sets the API to `rtl::rtmidi::api::alsa`.
- The `rt_select_input_ports()` function presents a list of input ports.
- Retrieved the API with `rtl::rtmidi::desired_api()`.

2. Create the MIDI input proxy. Pass the API to the `rtl::rtmidi_in` constructor. The three possible parameters are:

- The API. For *Linux*, they are *JACK*, *ALSA*, and, later, *Pipewire*. The default is "unspecified", which would invoke the fallback sequence that detects which APIs are available (currently JACK to ALSA).
- The client-name. Empty by default. For testing, the `-client name`.
- The input-queue size. Defaults to 100 MIDI messages.

3. Open the selected API. `rtl::rtmidi_in::open_midi_api()` detects the compiled APIs and tries to create the desired one.

4. Create the actual MIDI input. `rtl::midi_alsa_in()` with default client-name "rtl66 in".

5. Set ALSA data structure to defaults. Initializes `rtl::midi_alsa_data` to useless defaults.

6. Set up the ALSA client. The call sequence is:

- `rtl::midi_alsa_in::initialize()`
- `rtl::midi_alsa::impl_initialize()`

The ALSA sequencer handle (`snd_seq_t`) gets set up. Also called the "client" handle, not to be confused with "port" handles.

7. Set the tempo and PPQN. `rtl::midi_alsa::impl_set_tempo()`. Sets the default tempo and PPQN.

8. Get the number of ports. The call sequence is:

- `rtmidi_in::get_port_count()` calls
- Get the `midi_api` pointer returned by `rtl::rtmidi::rt_api_ptr()`.
- Call `midi_alsa_in::get_port_count()` via this pointer.
- `rtl::midi_alsa::impl_get_port_count()` for MIDI input.
- This function also gets the port capabilities via ALSA.
- Call the static function `rtl::get_port_info()` (in `midi_alsa.cpp`) with a port value of `-1`, which indicates "all ports".

9. Open the input port. The call sequence is:

- `rtl::rtmidi_in::open_port()`.
- `rtl::midi_alsa_in::open_port()`.
- `rtl::midi_alsa::impl_open_port()`.
- `rtl::get_port_info()`, this time for port 0.

10. **Get port information.** `rtl::get_port_info()` iterates through all input ports, retrieving port information (wasteful!) until it gets the proper kind of port where the index of the port matches the desired port number. .
11. **Create the port.** `rtl::rtmidi_in::open_port()` then gets the sender and receiver addresses, sets the client, the port, the capabilities, type, and number of channels in the port (16), and the port's name. (EMPTY!!!!!!) Optionally sets time-stamping information.
12. **Port subscription.** `midi_alsa::impl_subscription()` is called to set up ALSA port subscription, a fancy way to connect ports.

If ALSA were not specified, then `rtl::rtmidi::fallback_api()` would be called to try JACK first, then ALSA. (Later, PipeWire might be the first choice).

4.1.3 Walk-throughs / RtMidi API / cbmidiin

`tests/cbmidiin` is like `tests/qmidiin`, except that it passes the `midi::message` to a short callback function that simply prints the message. The callback is called in `midi_alsa_handler()` and `jack_process_in()` (other APIs not ready).

4.1.4 Walk-throughs / RtMidi API / midiclock

`tests/midiclock` tests the emitting and reception of MIDI clock events. The build of this test creates two versions of the same application: `tests/midiclock_in` and `tests/midiclock_out`. Run `tests/midiclock_in` first, read the instructions, pick a port (any port, 0 is fine), and then run `tests/midiclock_out`. Pick the port named `rtl-xxxx:rtl66 midi in 0`, where "xxxx" is "alsa" or "jack", and observe that both console windows reflect clock activity.

4.1.5 Walk-throughs / RtMidi API / midiout

`tests/midiout` tests sending a single tone. Once the desired output MIDI port is found, an `rtl::rtmidi_out` object is created. Then a series of `midi::message` items are created and sent by `rtl::rtmidi_out::send_message()`. The messages sent are a program change, a quarter frame, a control change, and a note on, followed by a 500 ms sleep, a note off, etc. The result should be one short note played.

4.1.6 Walk-throughs / RtMidi API / midiprobe

`tests/midiprobe` simply lists the APIs supported and active, and the ports. The former uses the `rtmidi` classes, and the latter uses the `masterbus` class.

4.1.7 Walk-throughs / RtMidi API / sysextest

`tests/sysextest` ... TODO

4.1.8 Walk-throughs / RtMidi API / test_c_api

`tests/test_c_api` does a very basic test of the "C" API. The input and output `RtMidiWrapper` objects are created internally and accessed by static pointers. Currently the only tests get the port count from each pointer, close the input and output ports, and then frees the allocated wrappers.

TODO

4.2 Walk-throughs / Master Bus API

4.2.1 Walk-throughs / Master Bus API / midiprobeex

`tests/midiprobeex` lists the APIs supported and active, and the ports, and uses the masterbus paradigm.

MORE TO COME

4.3 Supported MIDI APIs

Rtl66 supports the following MIDI APIs:

- PipeWire. (Planned, not yet implemented).
- JACK.
- ALSA.
- Windows Multi-Media.
- Mac OSX Core.
- Web MIDI.
- Dummy.

Audio APIs (derived from RtAudio) are planned.

5 MIDI Meta and System Exclusive Events

This section describes how the *Rtl66* library handles MIDI Meta and SysEx events. There are many intricacies to work with. It also covers, concisely, MIDI operations not yet implemented, but important to keep in the "to do" queue.

Also helpful is additional information about other MIDI events in the *Seq66* manual, section *MIDI Formats* [11].

5.1 MIDI Meta Events

Meta events are non-MIDI data of various sorts consisting of a fixed prefix, an event type, a length field, and the meta event data. Meta events are *never* sent to a device. *Rtl66* tries to load, store,

and write all meta events, even where it cannot use the meta data.

0xFF + meta_type + v_length + event_data_bytes

- meta_type is 1 byte, expressing one of the meta event types shown below.
- v_length is length of meta event data, a variable length ("varinum") value.
- event_data_bytes is the actual event data.

Here, we summarize the MIDI meta events data. Two-character names represent a single byte; *len* represents a variable-length value.

1. FF 00 02 ss ss. Sequence Number.
2. FF 01 len text. Text Event.
3. FF 02 len text. Copyright Notice.
4. FF 03 len text. Sequence/Track Name.
5. FF 04 len text. Instrument Name.
6. FF 05 len text. Lyric.
7. FF 06 len text. Marker.
8. FF 07 len text. Cue Point.
9. FF 08 len text. Program Name.
10. FF 09 len text. Port Name.
11. FF 2F 00. End of Track.
12. FF 51 03 tt tt tt. Set Tempo, us/qn.
13. FF 54 05 hr mn se fr ff. SMPTE Offset.
14. FF 58 04 nn dd cc bb. Time Signature.
15. FF 59 02 sf mi. Key Signature.
16. FF 7F len data. Sequencer-Specific.

We need to make sure we read, save, and restore all of the items above, even if *Rtl66* doesn't use them. Some are deprecated in the MIDI standard. The next sections describe the events that *Rtl66* tries to handle. These are:

- Generic Meta Messages.
- Sequence Number (0x00).
- Track Name (0x03).
- Other text events(0x04 to 0x09).
- End-of-Track (0x2F).
- Set Tempo (0x51).
- Time Signature (0x58).
- Key Signature (0x59).
- Sequencer-Specific (0x7F) (handled differently in *Rtl66*).
- System Exclusive (0xF0).

Also, all the text events should be handled by *Rtl66*, but a lot more testing is needed.

5.1.1 Generic Meta Messages

FF nn len data...

Any meta message not explicitly handled is read and is stored in a track. Nothing else is done with it. It can be written back out to a MIDI file as is.

5.1.2 Track/Sequence Number (0x00)

```
FF 00 02 ss ss
FF 00 00
```

This optional event must occur at the beginning of a track, before any non-zero delta-times, and before any transmittable MIDI events. It specifies the number of a sequence. The second form, lacking the "ss ss" (sequence number bytes), the track's location in the file is use. *Rtl66* will read that form, but convert it to the first form.

5.1.3 Track/Sequence Name (0x03)

```
FF 03 len text
```

If in a format 0 track, or the first track in a format 1 file, the name of the sequence (song). Otherwise, the name of the track.

5.1.4 End of Track/Sequence (0x2F)

```
FF 2F 00
```

This event is *not optional*. It is included so that an exact ending point may be specified for the track, so that it has an exact length, which is necessary for tracks which are looped or concatenated.

5.1.5 Set Tempo Event (0x51)

The MIDI Set Tempo meta event sets the tempo of a MIDI sequence in terms of the microseconds per quarter note. This is a meta message, so this event is never sent over MIDI ports to a MIDI device. After the delta time, this event consists of six bytes, with three tempo bytes.

```
FF 51 03 tt tt tt      Example: FF 51 03 07 A1 20
```

1. 0xFF is the status byte that indicates this is a Meta event.
2. 0x51 the meta event type that signifies this is a Set Tempo event.
3. 0x03 is the length of the event, always 3 bytes.

4. The remaining three bytes carry the number of microseconds per quarter note. For example, the three bytes above form the hexadecimal value 0x07A120 (500000 decimal), which means that there are 500,000 microseconds per quarter note.

Since there are 60,000,000 microseconds per minute, the event above translates to: set the tempo to $60,000,000 / 500,000 = 120$ quarter notes per minute (120 beats per minute).

This event normally appears in the first track. If not, the default tempo is 120 beats per minute. This event is important if the MIDI time division is specified in "pulses per quarter note", which does not itself define the length of the quarter note. The length of the quarter note is then determined by the Set Tempo meta event.

Representing tempos as time per beat instead of beat per time allows absolutely exact synchronization with a time-based sync protocol such as SMPTE time code or MIDI time code. This amount of accuracy in the tempo resolution allows a four-minute piece at 120 beats per minute to be accurate within 500 usec at the end of the piece.

5.1.6 Time Signature Event (0x58)

After the delta time, this event consists of seven bytes of data:

```
FF 58 04 nn dd cc bb
```

The time signature is expressed as four numbers. **nn** and **dd** represent the numerator and denominator of the time signature as it would be notated. The denominator is a negative power of two: 2 represents a quarter-note, 3 represents an eighth-note, etc. The **cc** parameter expresses the number of MIDI clocks in a metronome click. The **bb** parameter expresses the number of notated 32nd-notes in a MIDI quarter-note (24 MIDI Clocks). Example:

```
FF 58 04 04 02 18 08
```

1. 0xFF is the status byte that indicates this is a Meta event.
2. 0x58 the meta event type that signifies this is a Time Signature event.
3. 0x04 is the length of the event, always 4 bytes.
4. 0x04 is the numerator of the time signature, and ranges from 0x00 to 0xFF.
5. 0x02 is the log base 2 of the denominator, and is the power to which 2 must be raised to get the denominator. Here, the denominator is 2 to 0x02, or 4, so the time signature is 4/4.
6. 0x18 is the metronome pulse in terms of the number of MIDI clock ticks per click. Assuming 24 MIDI clocks per quarter note, the value here ($0x18 = 24$) indicates that the metronome will tick every 24/24 quarter note. If the value of the sixth byte were $0x30 = 48$, the metronome clicks every two quarter notes, i.e. every half-note.
7. 0x08 defines the number of 32nd notes per beat. This byte is usually 8 as there is usually one quarter note per beat, and one quarter note contains eight 32nd notes.

If a time signature event is not present in a MIDI sequence, a 4/4 signature is assumed.

5.1.7 Key Signature Event (0x59)

After the delta time, this event consists of five bytes of data:

```
FF 59 02 ss kk
```

1. 0xFF is the status byte that indicates this is a Meta event.
2. 0x59 the meta event type that signifies this is a Key Signature event.
3. 0x02 is the length of the event, always 2 bytes.
4. ss is the number of the scale.
5. kk is 0 for a major scale, and 1 for a minor scale.

5.1.8 Text Events

Text events store an arbitrary number of bytes to represent a human-readable string. The characters should be limited to the ASCII character set.

```
FF nn len text
```

Here are the values of *nn* and the text they represent:

1. 0x01. Text Event. **FF 01 len text**. Supplies an arbitrary text string tagged to the track and time. It can be used for textual information which doesn't fall into one of the more specific categories below.
2. 0x02. Copyright. **FF 02 len text**. The text specifies copyright information for the sequence (song). This is usually placed at time 0 of the first track in the sequence (song).
3. 0x03. Track Name. **FF 03 len text**. See section [5.1.3 "Track/Sequence Name \(0x03\)"](#), page [23](#).
4. 0x04. Instrument name. **FF 04 len text**. The text names the instrument intended to play the contents of this track. Usually placed at time 0 of the track. Note that this meta-event is simply a description; MIDI synthesisers are not required (and rarely if ever) respond to it. This meta event is particularly useful in sequences prepared for synthesisers which do not conform to the General MIDI patch set, as it documents the intended instrument for the track when the sequence is used on a synthesiser with a different patch set.
5. 0x05. Lyric. **FF 05 len text**. The text gives a lyric intended to be sung at the given time. Lyrics are often broken down into separate syllables to time-align them more precisely with the sequence. See karaoke formats.
6. 0x06. Marker. **FF 06 len text**. The text marks a point in the sequence which occurs at the given time; for example "Third Movement".
7. 0x07. Cue Point. **FF 07 len text**. The text identifies a synchronization point which occurs at the specified time; for example, "Door slams".
8. 0x08. Program Name. **FF 08 len text**. The name of the program (i.e. patch) used to play the track. This may be different than the Sequence/Track Name. For example, if the name of your track is "Butterfly", and the track is played upon an electric piano patch, one may include a Program Name of "Electric Piano".
9. 0x09. Port Name. **FF 09 len text**. The name of the MIDI device (port) where the track

is routed. It replaces the "MIDI Port" meta event which some sequencers formally used to route MIDI tracks to various MIDI ports (to support more than 16 MIDI channels). Example: assume a MIDI interface that has 4 MIDI output ports. They are listed as "MIDI Out 1", "MIDI Out 2", "MIDI Out 3", and "MIDI Out 4". If you wished a particular track to use "MIDI Out 1" then you would put a Port Name Meta-event at the beginning of the track, with "MIDI Out 1" as the text. All MIDI events that occur in the track, after a given Port Name event, will be routed to that port. In a format 0 MIDI file, it would be permissible to have numerous Port Name events intermixed with MIDI events, so that the one track could address numerous ports. But that would likely make the MIDI file much larger than it need be. The Port Name event is useful primarily in format 1 MIDI files, where each track gets routed to one particular port.

5.2 Sequencer-Specific Meta-Events Format

This data, also known as **SeqSpec** data, provides a way to encode information that a specific sequencer application needs, while marking it so that other sequences can safely ignore the information.

```
FF 7F len id data
```

The first byte of the message is the "manufacturer ID".

5.2.1 Meta Events Handling

If the event status masks off to 0xF0 (0xF0 to 0xFF), then it is a real-time event. If the Meta event byte is 0xF7, it is called a "Sequencer-specific", or "SeqSpec" event. For this kind of event, then one to three manufacturer ID bytes and the length of the event are read.

Meta type:	varies	1 byte
Meta length:	varies	1 or more bytes

If the type of the **SeqSpec** (0xFF) meta event is 0x7F, parsing checks to see if it is one of the *Rtl66*-specific events. These events are tagged with various values that mask off to 0x242400nn. The parser reads the tag:

Prop tag:	0x242400nn	4 bytes
-----------	------------	---------

These tags provide a way to save and recover *Seq24/Rtl66* properties from the MIDI file: MIDI buss, MIDI channel, time signature, sequence triggers, and the key, scale, and background sequence to use with the track/sequence. Any leftover data for the tagged event is let go. Unknown tags are skipped.

If the type of the **SeqSpec** (0xFF) meta event is 0x2F, then it is the End-of-Track marker. The current time marks the length (in MIDI pulses) of the sequence. Parsing is done for that track.

If the type of the **SeqSpec** (0xFF) meta event is 0x03, then it is the sequence name. The "length" number of bytes are read, and loaded as the sequence name.

If the type of the **SeqSpec** (0xFF) meta event is 0x00, then it is the sequence number, which is read:

```
Seq number:    varies        2 bytes
```

Note that the sequence number might be modified latter to account for the current *Seq24* screenset in force for a file import operation. Any other **SeqSpec** type is simply skipped by reading the "length" number of bytes. The remaining sections simply describe MIDI meta events in more detail, for reference.

5.3 System Exclusive Event (0xF0)

System messages are used to send data such as patch parameters, sampler data, or a sequencer memory bulk dump. If the meta event status value is 0xF0, it is a "System-exclusive", or "SysEx" event. *Rtl66* stores these messages, but there are complications to be dealt with.

SysEx messages as sent and received are of the format:

```
F0 ID sysex data bytes F7
```

The ID is a manufacturer's ID. Normally a single byte, additional IDs can be represented by the sequence 00 xx yy, to allow 16384 additional manufacturer IDs. Here is the distribution of ID numbers:

	American	European	Japanese	Other	Special
1 byte ID:	01 - 1F	20 - 3F	40 - 5F	60 - 7C	7D - 7F
3 byte ID:	00 00 01 00 1F 7F	00 20 00 00 3F 7F	00 40 00 00 5F 7F	00 60 00 00 7F 7F	

"00" and "00 00 00" are not to be used. Currently *Rtl66* does not try to interpret the manufacturer ID.

There are three additional special ID codes that are *not* manufacturer-specific:

- **0x7D**. Used for private non-commercial purposes only. It has no standard format.
- **0x7E**. Represents a non-realtime universals system exclusive message. All MIDI devices can respond to it, although not immediately.
- **0x7F**. Represents a realtime universals system exclusive message. All MIDI devices can respond to it, immediately.

The format for 0x7E and 0x7F is the same:

```
F0H ID_number device_ID sub-ID#1 sub-ID#2 ... F7H
```

An ID number of 0x7F (the 'all call' ID) indicates all devices should respond. As for device ID, some devices might have multiple IDs.

For more information on these events, see section 5.4.1 "Universal SysEx Events (0xF0 0x7E and 0xF0 0x7F)", page 31.

Now, when encoded for storage in a MIDI file, the format of the bare message (first line) is padded so that it starts with a delta time, and includes the length (a variable-length quantity) of the SysEx including the final F7, but excluding the F0 and the length bytes.:

```
F0 ID sysex data bytes F7      (message)
delta F0 len ID sysex data bytes F7  (encoded)
```

SysEx events can be encoded in the following ways:

- **Single SysEx Message.**
- **Continuation Events.**
- **Escape Sequence.**

5.3.0.1 Single SysEx Message

A SysEx message that is *sent to a device* is not quite the same as a SysEx message that is *encoded in a MIDI file*. SysEx messages encoded in a MIDI file are preceded by a variable-length delta time, the byte F0, a variable-length length value, and the message, and a terminating F7, which *must* be present, and is counted in the length value. In the following example, the first line is the actual SysEx message (a General MIDI Enable message), and the second line is its encoding in a MIDI file, ignoring the preceding delta time:

```
F0 7E 00 09 01 F7      (message)
F0 05 7E 00 09 01 F7  (encoded)
```

5.3.0.2 Continuation Events

Some equipment needs the SysEx to be split into smaller chunks for processing. This could be accomplished with a number of smaller single-SysEx messages, but some manufacturers treat SysEx as if it supported running status (it does not). So the first message starts with F0, the next messages have only the SysEx data, and the last message ends with F7. However, when encoding in the MIDI file, each sub-packet begins with an F7. In between are the delta times to use to delay the sub-packets when sending to a slow device. An unencoded message with 3 packets is shown, with a bar for a separator for this discussion:

```
F0 43 12 00 | 43 12 00 43 12 00 | 43 12 00 F7
```

Encoded with a 200-tick delta time (81 48) between each message, the F7 being used as a *continuation* byte, with length values:

```
00 F0 03 43 12 00
81 48 F7 06 43 12 00 43 12 00
81 48 F7 04 43 12 00 F7
```

Since the first message is not terminated by F7 within the specified length, the next two F7s indicate a continuation. Note the F7 at the beginning of packets after the first one, and the F7 at the end of the last one.

5.3.0.3 Escape Sequence

An escape sequence is not SysEx, but it does use the F7 byte. It is used for encoding arbitrary bytes for messages such as Song Select. The first line shows such a message, and the second how it would be encoded.

```
F3 01
F7 02 F3 01
```

The format is F7 len <all bytes to be transmitted>.

So how to figure out what F7 means? Its interpretation is as follows:

- An F0 without a terminal F7 within the specified length is a Casio-style multi-packet message, and a flag (call it **multi**) should be raised to indicate this status.
- If F7 is encountered while **multi** is set, it is a continuation.
- If this continuation ends with F7, it is the last packet and **multi** should be cleared.
- If F7 is encountered while **multi** is *not* set, then the event is an escape sequence.

5.4 System Exclusive Sample Dump Standard

This standard is not yet implemented in *Rtl66*. It provides for for ampler data dumps. Five of the basic messages are generic handshaking messages (ACK, NAK, Wait, Cancel & EOF), are also used in other applications e.g. File Dump. The remaining messages are Dump Request, Dump Header, Data Packets, and a Sample Dump Extensions message. The data formats are given in hexadecimal.

Here we only summarize them.

5.4.0.1 Generic Handshaking Messages

In these descriptions, **pp** is the packet number.

- ACK: F0 7E <device ID> 7F pp F7. The first handshaking flag, meaning "Last data packet was received correctly. Send the next one." The packet number represents the packet being acknowledged as correct.

- NAK: F0 7E <device ID> 7E pp F7. The second handshaking flag, meaning "Last data packet was received incorrectly. Please resend." The packet number represents the packet being rejected.
- CANCEL: F0 7E <device ID> 7D pp F7. The third handshaking flag. It means "Abort dump." The packet number represents the packet on which the abort takes place.
- WAIT: F0 7E <device ID> 7C pp F7. The fourth handshaking flag, meaning "Do not send any more packets until told to do otherwise." This is important for systems in which the receiver (such as a computer) may need to perform other operations (such as disk access) before receiving the remainder of the dump. An ACK will continue the dump while a Cancel will abort the dump.
- EOF: F0 7E <device ID> 7B pp F7. A new generic handshaking flag added for the File Dump extension.

5.4.0.2 Data Messages

- DUMP HEADER. F0 7E <device ID> 01 ss ss ee ff ff ff gg gg gg hh hh hh ii ii ii jj F7.
 - ss = Sample number (LSB first)
 - ee = Sample format (# of significant bits from 8-28)
 - ff = Sample period (1/sample rate) in nanoseconds (LSB first)
 - gg = Sample length in words (LSB first)
 - hh = Sustain loop start point word number (LSB first)
 - ii = Sustain loop end point word number (LSB first)
 - jj = Loop type (00 = forward only, 01 = backward/forward, 7F = Loop off)
- DUMP REQUEST. F0 7E <device ID> 03 ss ss F7, where ss = Requested sample, LSB first. Upon receiving this message, the sampler should check to see if the requested sample number falls in a legal range. If it is, the requested sample becomes the current sound number and is dumped to the requesting master following the procedure outlined in the documentation. If it is not within a legal range, the message should be ignored.
- DATA PACKET. F0 7E <device ID> 02 kk <120 bytes> ll F7, where kk = Running packet count (0-127); and ll = Checksum (XOR of 7E <device ID> 02 kk <120 bytes>). The total size of a data packet is 127 bytes. This is to prevent MIDI input buffer overflow in machines that may want to receive an entire message before processing it. 128 bytes, or 1/2 page of memory, is considered the smallest reasonable buffer for modern MIDI instruments.

There are some more scenarios in the MIDI 1.0 detailed specification document, including:

- Sample dump extensions and a description of the sample dump scenario, too detailed to include here.
- Device inquiry and response. Messages used for device identification, categorized non-realtime SysEx general information messages.
- File dump provides a protocol for transmitting files from one computer to another using MIDI. Two primary motivations for this protocol: transmitting MIDI Files (and tempo maps) between computers and small microcomputer "boxes"; and transmitting files of *any* type between two computers of different types. The filename is sent with the file. The protocol includes a request, header, data packet, hand-shaking flags,

- MIDI Tuning.

There is so much more to look into. Sigh.

5.4.1 Universal SysEx Events (0xF0 0x7E and 0xF0 0x7F)

As noted earlier, 0x7E and 0x7F provide for non-realtime and realtime SysEx message. Immediately following these value is the "channel", which can be a manufacturer's ID or any value ranging from 0x00 to 0x7F (which is a wild-card for "all devices").

```
F7 7E channel subid1 subid2 data bytes F7      (non-realtime)
F7 7F channel subid1 subid2 data bytes F7      (realtime)
```

If `channel` is a device ID. If its value is F7, this is a global broadcast that all devices should heed. The `subid1` and `subid2` could be something like "01 00" for a long-form (full frame) time-code message.

A detailed description is beyond the scope of this document. Some messages supported by these messages are the *MIDI Master Volume*, *MIDI Full Frame*, and the *General MIDI System Enable/Disable* messages.

An interesting subset of these messages is *MIDI Show Control* (MSC):

```
F7 7F deviceid 02 commandformat command data F7      (MSC)
```

Some simple controls can be included in the 'ctrl' file's set of macros.

5.4.1.1 Rtl66 SysEx Handling

Rtl66 is slowly gaining support for reading, storing, and sending SysEx events within a sequence. *Rtl66* warns if the terminating 0xF7 SysEx terminator is not found at the expected length. Also, some malformed SysEx events have been encountered, and those are detected and handled as well.

The format of a bare (i.e. not encoded in a MIDI file) SysEx message is like the following, complex case:

```
F0 manid devid modelid direction address  data checksum terminator
F0 0x41 0x10 0x42 0x12 0x40007F 0x00 0x41 0xF7
```

The "manid" is a manufacturer's identifier. 0x41 is Roland, 0x24 is Hohner. 0x00 is used for adding two more codes to greatly expand the ID list. 0x7E is used to denote a non-realtime message, and 0x7F denotes a realtime message. The "devid" is the device identifier (e.g. 0x10 for Roland devices). The "devid" indicates which devices will accept the SysEx message. The "modelid" is the model identifier (0x42 for most GS synths). The "direction" is 0x12 for sending information or 0x11 for requesting information. The "address" is a 3-byte value on which the SysEx message should act.

Devices provide an address map to define what is at each address. For example, this address might define a "GS Reset", which performs an initialization. The "data" is either the data to send (one or more bytes), or the size of the data we are requesting. Some devices will include a "checksum" for data integrity. All SysEx messages end with an F7 byte.

6 References

The *Rtl66* references list.

References

- [1] ALSA team. *Advanced Linux Sound Architecture (ALSA) project homepage*. <http://www.alsa-project.org/>. ALSA tools through version 1.0.29. 2015.
- [2] JACK team. *JACK Audio Connection Kit*. <http://jackaudio.org/>. 2015.
- [3] Ableton Live Link. *Ableton Link (transport)*. <https://www.ableton.com/en/link/>, <https://github.com/Ableton/link/>. 2022.
- [4] JACK Audio. *New Session Manager, an offshot of Non Session Manager*. <https://github.com/jackaudio/new-session-manager> 2021.
- [5] Pimpl. *The Pimpl (pointer to implementation) paradigm*. <https://en.cppreference.com/w/cpp/language/pimpl.html> Also called "compilation firewall", "handle body", and "cheshire cat" idioms.
- [6] PortMedia team. *Platform Independent Library for MIDI I/O*. <http://portmedia.sourceforge.net/>. 2010.
- [7] Gary P. Scavone. *RtAudio home page*. <https://caml.music.mcgill.ca/~gary/rtaudio/> 2023.
- [8] Gary P. Scavone. *The RtMIDI Tutorial*. <https://www.music.mcgill.ca/~gary/rtmidi/>. 2023.
- [9] Seq24 Team. *The home site for the Seq24 looping sequencer*. <http://www.filter24.org/seq24/download.html>. 2010.
- [10] Chris Ahlstrom. *A reboot of the Seq24 project as "Sequencer64"*. <https://github.com/ahlstromcj/sequencer64/>. 2015-2021.
- [11] Chris Ahlstrom. *A reboot of the Seq24 project as "Seq66"*. <https://github.com/ahlstromcj/seq66/>. 2015-2022.

Index

ABC, [7](#)

ALSA, [17](#)

callback

 error, [6](#)

 input, [7](#), [8](#), [11](#)

functor, [6](#)

history, [5](#)

issue

 masterbus and ALSA, [17](#)

 masterbus and JACK, [17](#)

namespace

 global, [6](#)

 midi, [5](#), [10](#)

 rtl, [5](#)

paradigm

 masterbus, [16](#)

 RtMidi, [16](#)

rtl, [5](#)